Lab 3 - Synthesizing an RTL Design

On completing Lab 3 one will get a hands-on experience in controlling PMOD pins of the Kria KV260 board. We will connect the PMOD to some LEDs and write software to control them from the Serial Terminal.

Pre-Requisite

1. Successful completion of Lab1 and Lab2.
2. Basic understanding of C programming and digital circuits.
3. Five LEDs, five 200Ohm resistors, one Bread-board, six male-male jumper wires.

Section 1: Creating Harware Platform on Vivado 2024.1

Create Vivado Project

1. Launch the Vivado 2024.1 on your PC.
2. Click on **Create Project** icon from the **Quick Start** menu.
3. Click on **Next**, then give name to the project and choose suitable project location. Make sure to check-on the **Create project subdirectory** option.
4. Choose **RTL Project**, **Do not specify sources at this time** and Click on **Next**.
5. Click on **Boards** and search for **"Kria KV260 Vision AI Starter Kit"**, select it and click **Next**.
6. Click on **Finish**.

Create Block Design

1. Click on **Create Block Design** from the **Flow Navigator** pane.
2. Give suitable name(e.g. led_Control) to the design and click on **OK**.
3. Click on the "+" button(Add IP) in the **Diagram** window. Search for the **Zynq Utrascale+ MPSoC** IP, and select it by pressing enter or double clicking it. Refer figure 1.

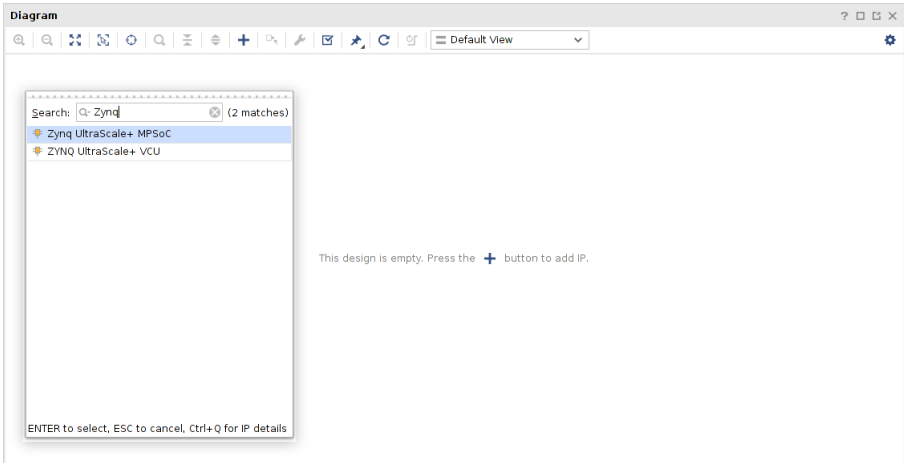


Figure 1: Search IP

4. Notice the **Designer Assistance available** pops up. Click on **Run Block Automation**.

5. In **Run Block Automation** window, make sure **Apply Board Preset** option is checked. Click on **OK**.

6. Double click on **Zynq Utrascale+ MPSoC** IP. Then click on the **UART1** block from the **PS Ultrascale+ Block Design**. Refer figure 2.

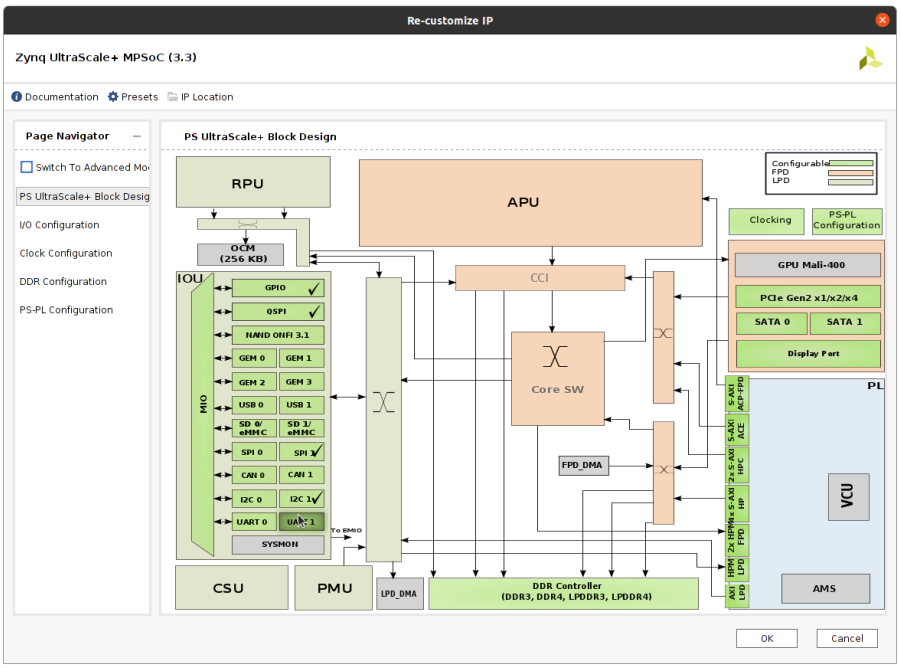


Figure 2: Re-customize IP

7. Then check **UART1** box and choose **MIO 36..37** in the **I/O**. Refer figure 3. Then click **OK**. This enables the UART capability of the Processing System.

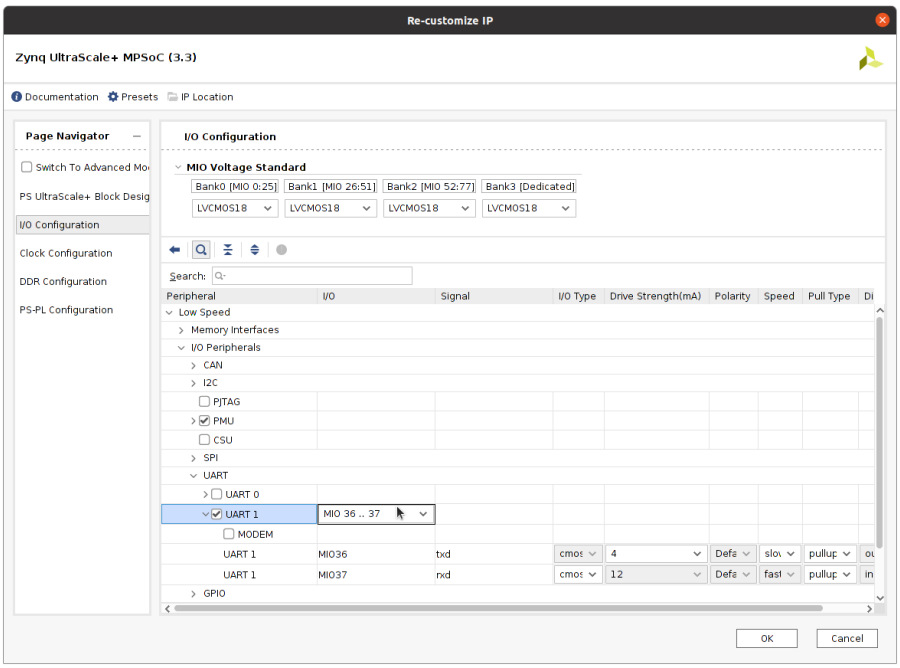


Figure 3: UART Configuration

8. Click on the "+" button(Add IP) again. Search for the **AXI GPIO** IP, and select it by pressing enter or double clicking it.

9. Notice the **Designer Assistance available** pops up. Click on **Run Connection Automation**.

10. In **Run Connection Automation** window, make sure **All Automation** option is checked. Click on **OK**.

11. Notice the **Designer Assistance available** pops up. Click on **Run Connection Automation**.

12. Again in **Run Connection Automation** window, make sure **All Automation** option is checked. Click on **OK**. Refer figure 4.

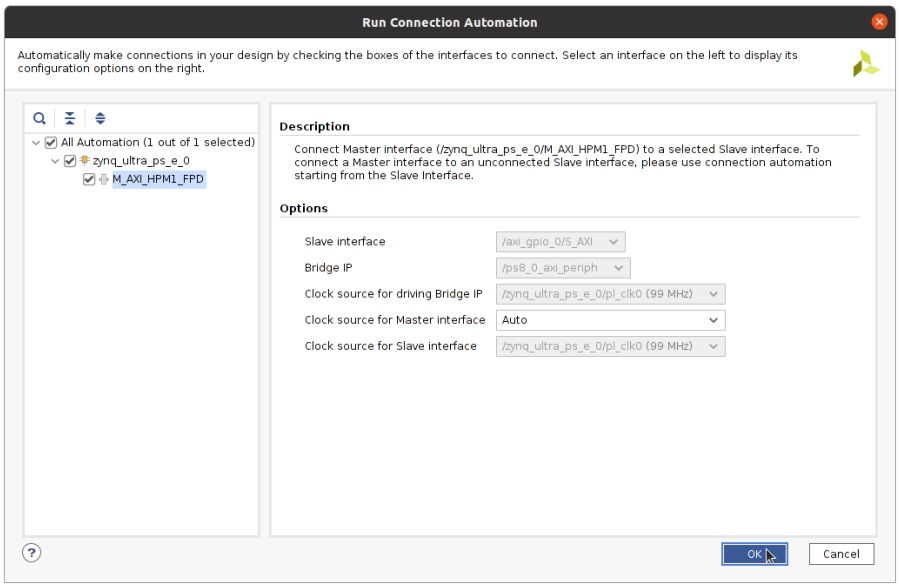


Figure 4: Run Connection Automation Window

13. Click on **Regenerate Layout** option. Refer figure 5.

Figure 5: Regenerate Layout Icon

14. Double click on **AXI GPIO** IP to customise it. Change the **GPIO Width** to 5. Refer figure 6.

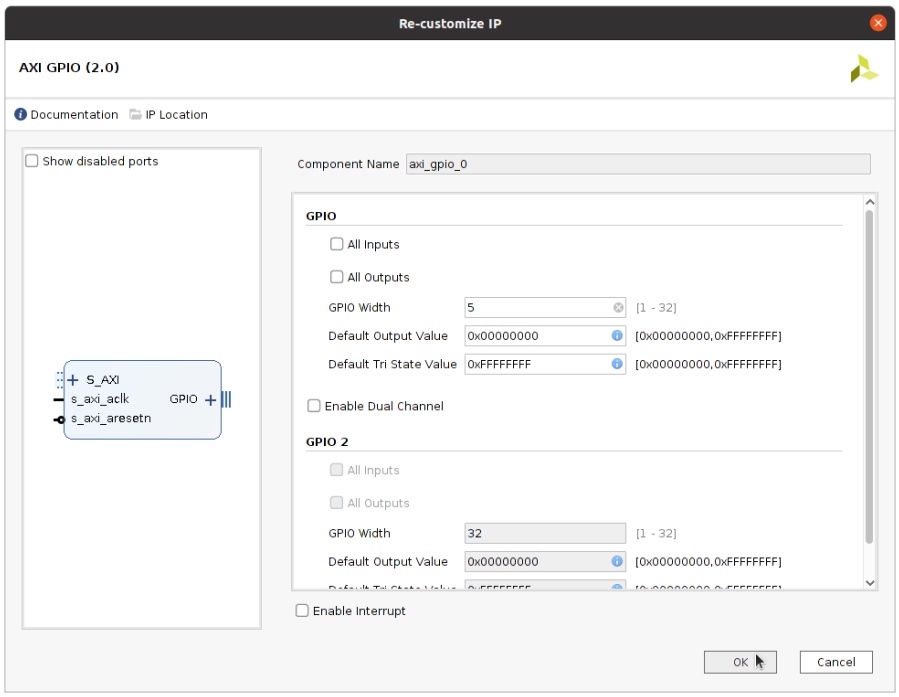


Figure 6: Customizing AXI GPIO IP

15. Press **F6** on your keyboard to *"Validate"* the *"Block Design"*. And save by pressing *"Ctrl + S"* on your keyboard.

16. Your block design should look like 7. The **gpio_rtl** is external pin that we will connect to the PMOD pins via **IO-Planning**.

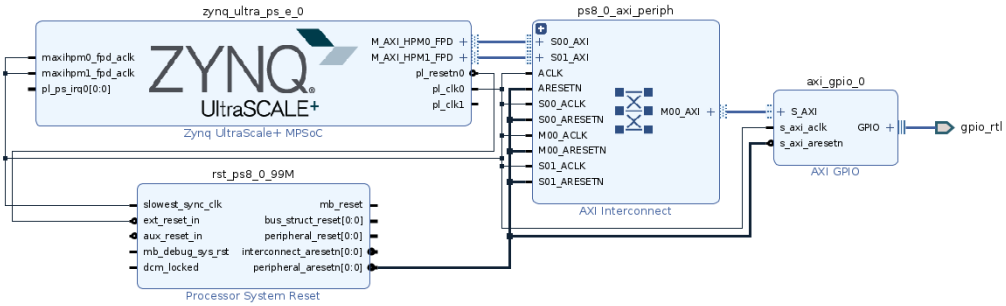


Figure 7: Complete Block Design

17. Go to **Sources** tab and Right-Click on the **.bd** file(e.g. led_Control.bd) and click **Create HDL Wrapper**. Refer figure 8.

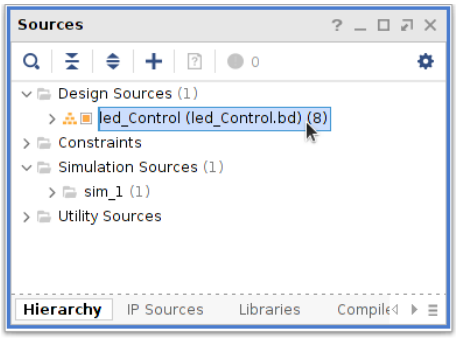


Figure 8: Create HDL Wrapper

18. Select **Let Vivado manage wrapper and auto-update** option and click **OK**. Refer figure 9.

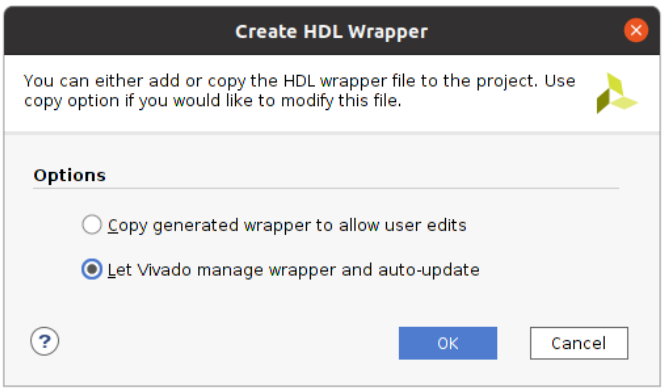


Figure 9: Create HDL Wrapper

Export Hardware Platform

1. Click on **Run Synthesis** option from the **Flow Navigator** pane under **SYNTHESIS**.

2. When synthesis is completed, choose the **Open Synthesized Design** option and click **OK**. Refer figure 10.

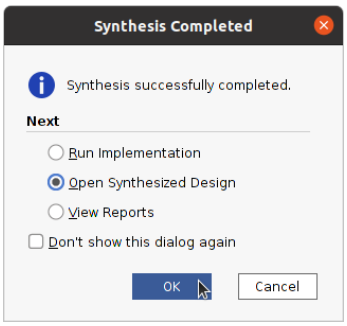


Figure 10: Synthesis Completed Window

3. Navigate to **I/O Ports** tab. Assign the ports to appropriate package pins like figure 11. Also make sure to choose **LVCMOS33** as **I/O Std** for all ports.

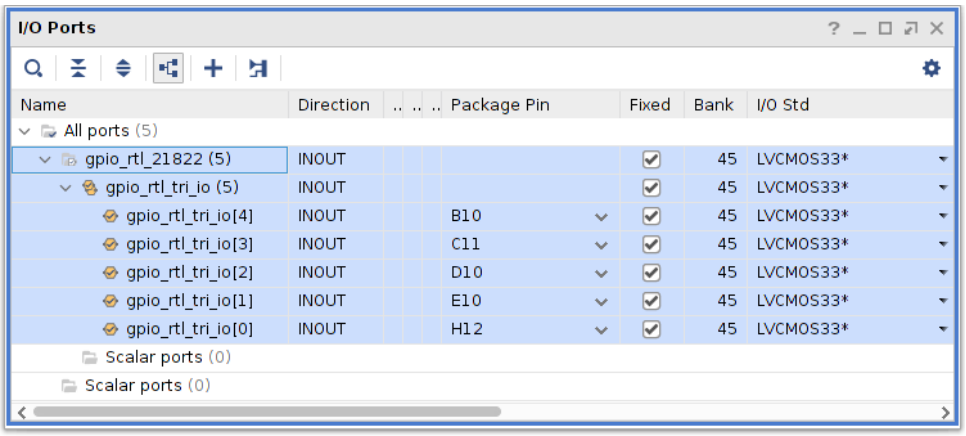


Figure 11: I/O Planning

4. Press (Ctrl+S) to save the configuration. Give the constrain file a suitable name. Refer figure 12.

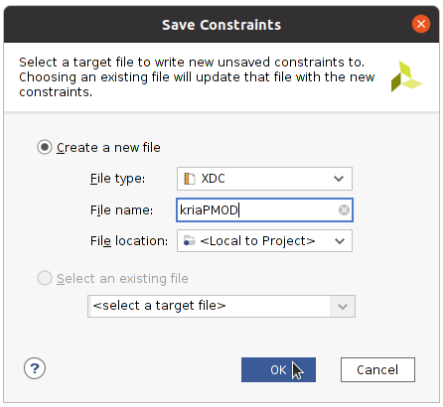


Figure 12: Saving Constrain File

5. Click on **Generate Bitstream** option from the **Flow Navigator** pane under **PROGRAM AND DEBUG**.

6. Click on **Yes**. This will automatically run synthesis and implementation steps. Refer figure 13.

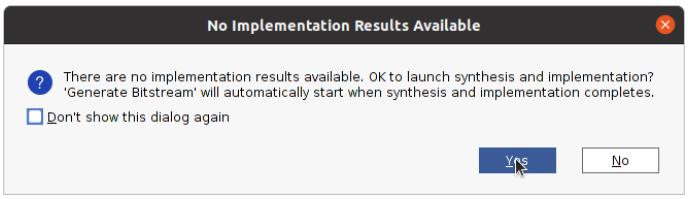


Figure 13: Generate Bitstream

7. Next click on **OK** in the **Launch Runs** window. Make sure that **Launch Runs on local host** option is checked. Refer figure 14.

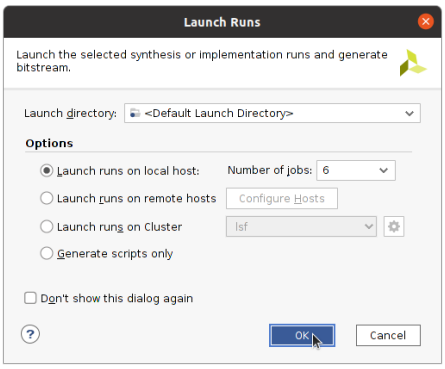


Figure 14: Generate Bitstream

8. When Bitstream Generation is completed, click on **Cancel**.

9. Click on **File** on the top panel of the Vivado Window. Go to **Export** then **Export Hardware**.

10. **Export Hardware Platform** window will pop up. Click on **Next**.

11. Choose the **Include bitstream** option and click on **Next**. Refer figure 15.

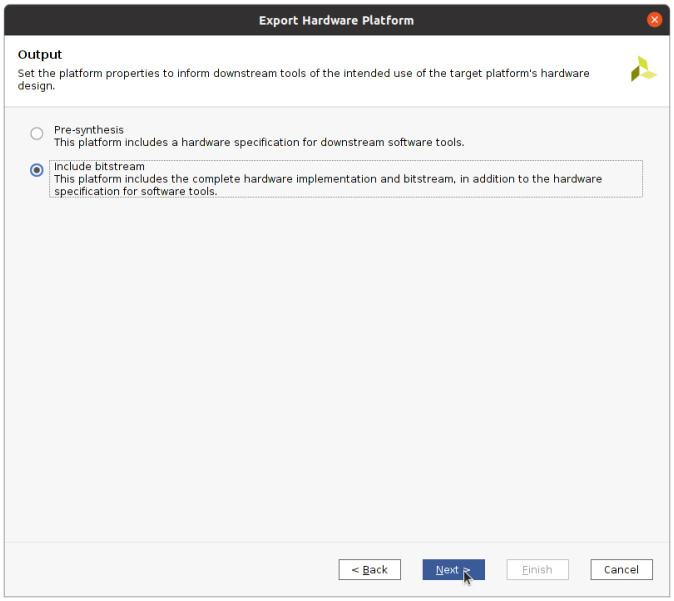


Figure 15: Export Hardware Platform

12. Next window will show the **.XSA** file name and location. Remember this file's location, because we will need it for next steps. Then click on **Next**. Refer figure 16.

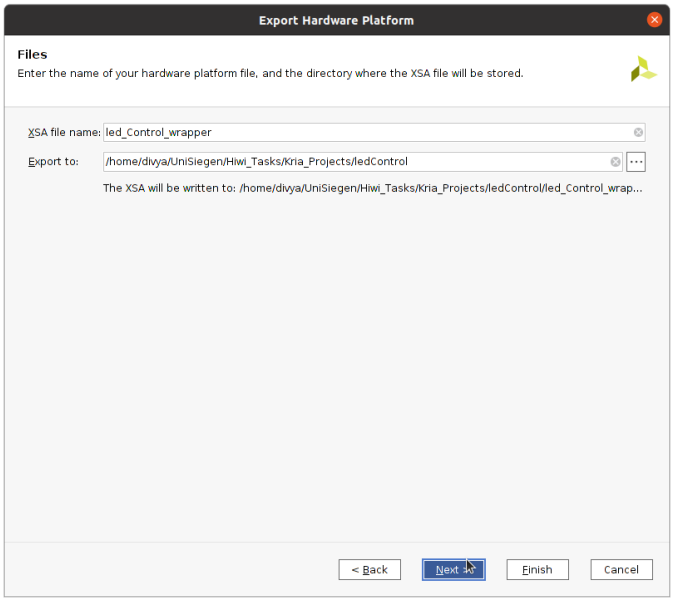


Figure 16: Choose Hardware Platform Location

13. Click on **Finish**.

Section 2: Creating Application Project on Vitis 2024.1

1. Launch Vitis normally or launch it from Vivado by going to **Tools** then **Launch Vitis IDE**. Vitis window looks as figure 17.

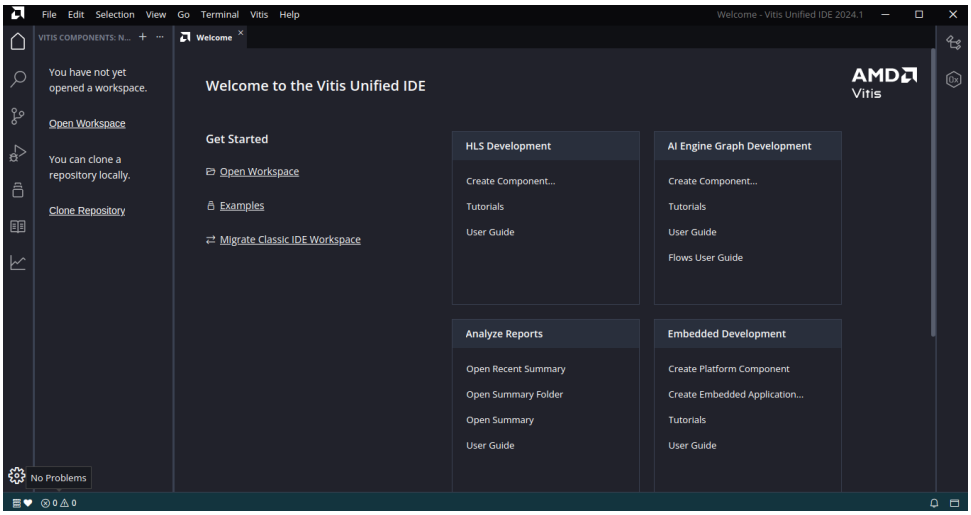


Figure 17: Vitis Window

2. Click on **Create Platform Component** option under **Embedded Development**.

3. Provide the **Component Name** and **Component Location** where the project should be created. Then click on **Next**. Refer figure 18.

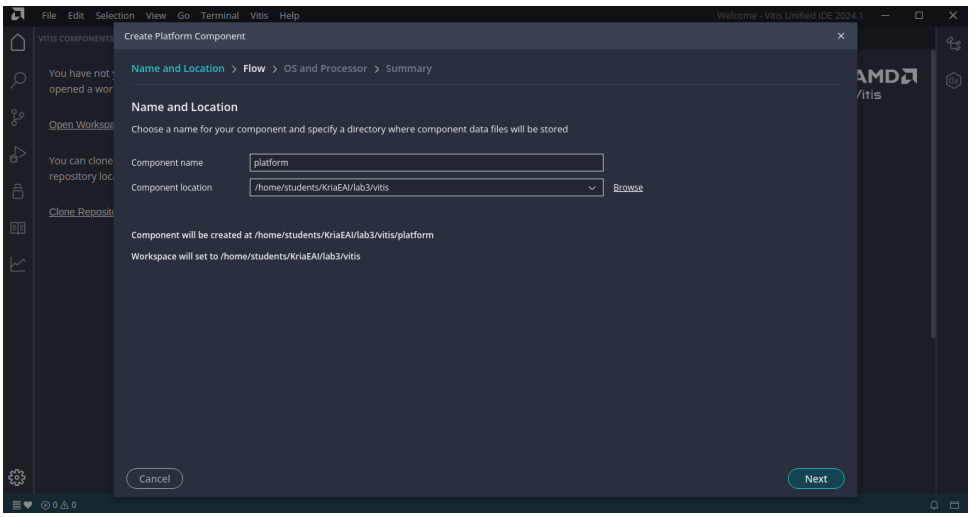


Figure 18: Component Name

4. Click on **Browse** and select the previously exported hardware design(XSA). Then click on **Next**. Refer figure 19.

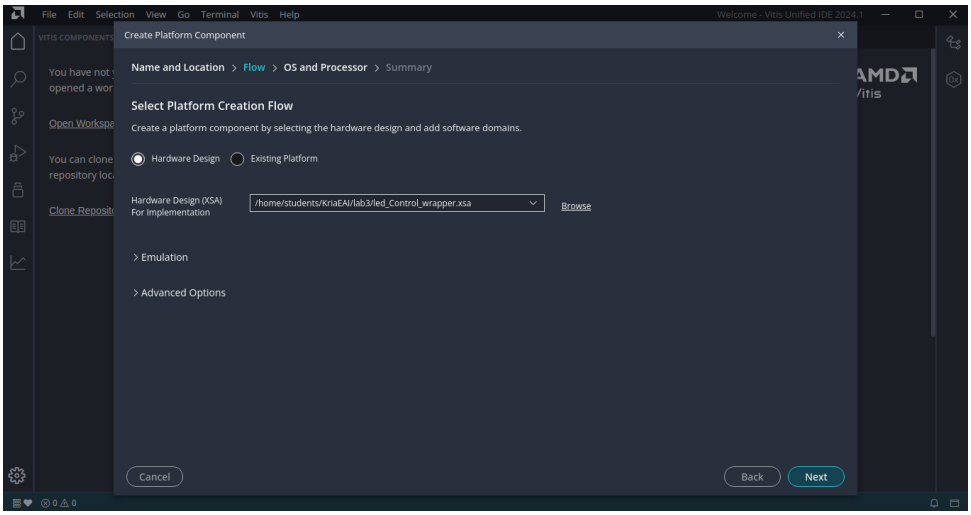


Figure 19: Browse Hardware Design

5. The next window will show the Operating System and Processor information. Then select **Next**.

6. Then all the information related to platform will be shown in the window. Then select **Finish**. Refer figure 20.

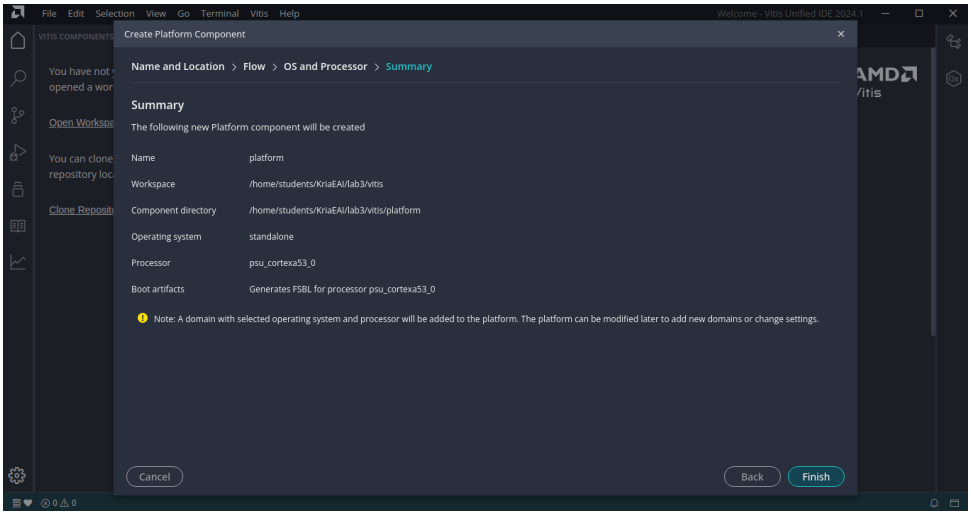


Figure 20: Platform Component Summary

7. After the platform project is created and visible on the work-space, build it using the hammer icon in the left side. After successful compilation of the platform it will show green tick. Refer figure 21.

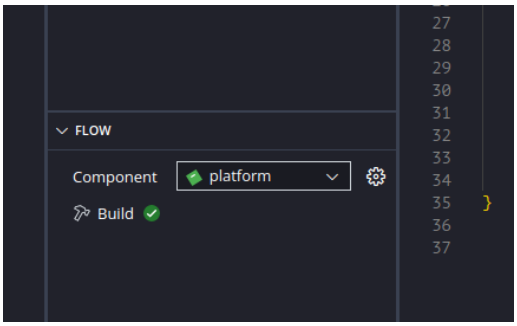


Figure 21: Building Platform Component

8. On the left pane window select **Examples** and then select **Empty Application**. Refer figure 22.

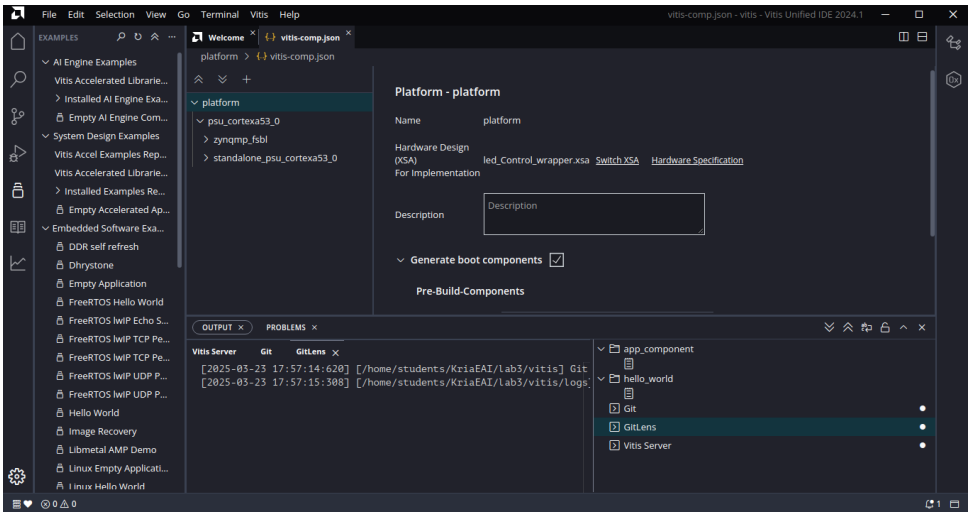


Figure 22: Empty Application Creation

9. Then select **Create Application Component from Template**. Then give the application a name(e.g. ledControl). Click **Next**. Refer figure 23.

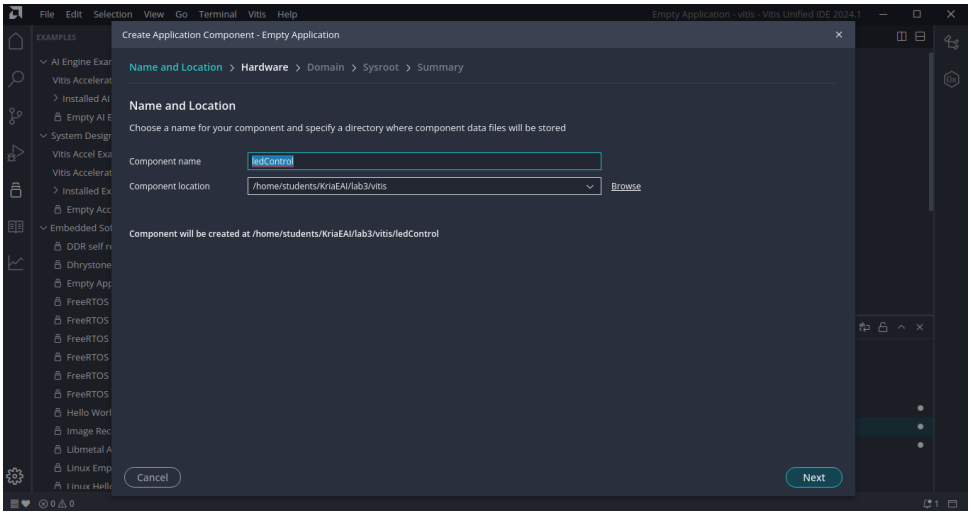


Figure 23: Empty Application Creation: Name

10. Select the platform named **platform** that we created earlier. Click on **Next**.

11. Then choose the default Domain, appearing in the option and the click on **Next**.

12. Click on **Finish**.

13. Now expand the application component(e.g ledControl) and expand **Sources** option then right-click on the **src** and add a new C source file(e.g ledControl.c). Click on **OK**. Refer figure 24.

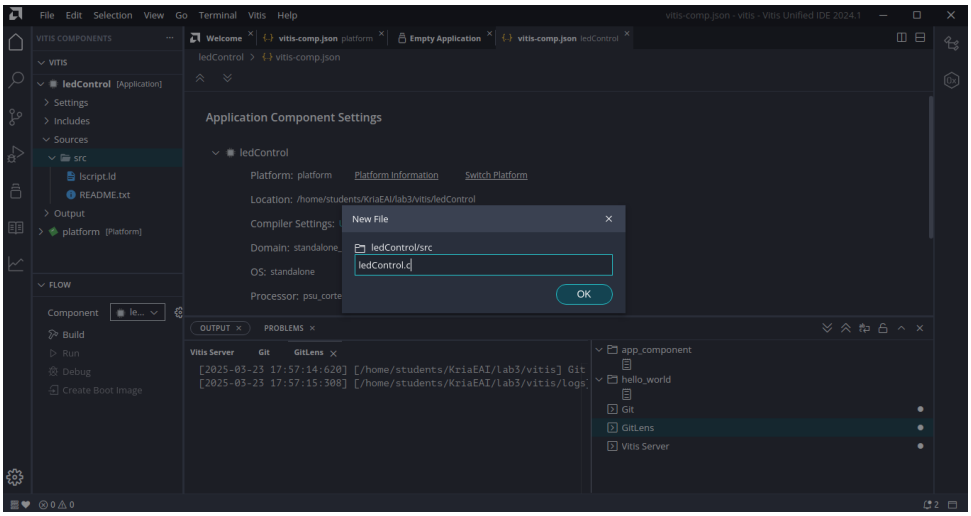
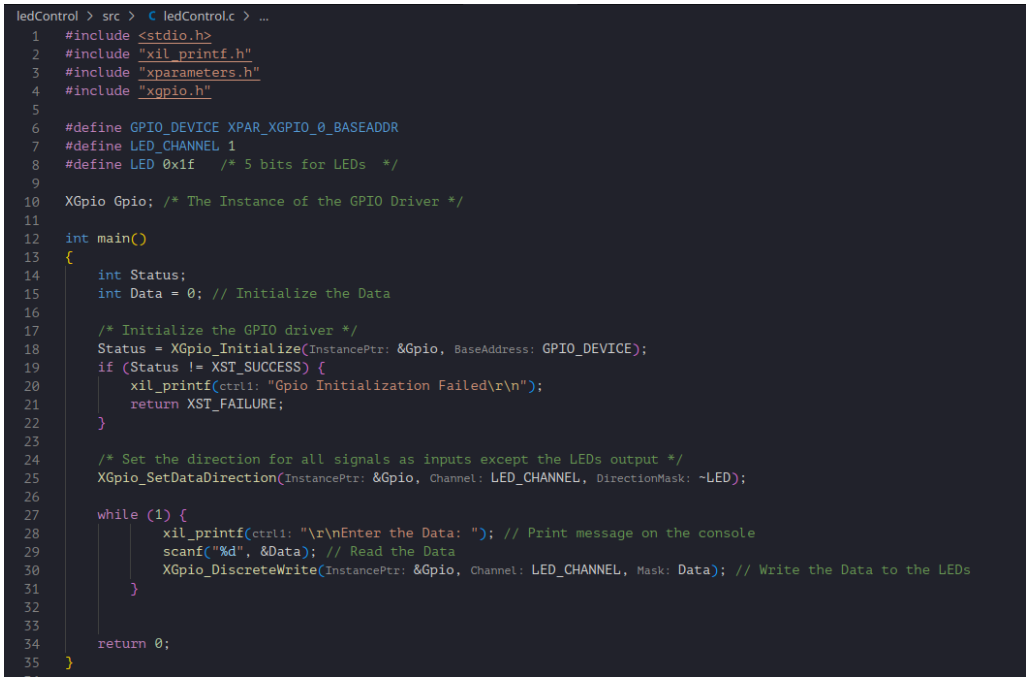


Figure 24: Adding C source file

14. Write the following **Lab3_ledControl.c** program from the provided source folder "Lab3-SynthesizingRTLDesign", figure 25. Save the program by pressing "Ctrl+s".



```
ledControl > src > C ledControl.c > ...
#include <stdio.h>
#include "xil_printf.h"
#include "xparameters.h"
#include "xgpio.h"

#define GPIO_DEVICE XPAR_XGPIO_0_BASEADDR
#define LED_CHANNEL 1
#define LED 0x1f   /* 5 bits for LEDs  */

XGpio Gpio; /* The Instance of the GPIO Driver */

int main()
{
    int Status;
    int Data = 0; // Initialize the Data

    /* Initialize the GPIO driver */
    Status = XGpio_Initialize(InstancePtr: &Gpio, BaseAddress: GPIO_DEVICE);
    if (Status != XST_SUCCESS) {
        xil_printf(ctrl1: "Gpio Initialization Failed\r\n");
        return XST_FAILURE;
    }

    /* Set the direction for all signals as inputs except the LEDs output */
    XGpio_SetDataDirection(InstancePtr: &Gpio, Channel: LED_CHANNEL, DirectionMask: ~LED);

    while (1) {
            xil_printf(ctrl1: "\r\nEnter the Data: "); // Print message on the console
            scanf("%d", &Data); // Read the Data
            XGpio_DiscreteWrite(InstancePtr: &Gpio, Channel: LED_CHANNEL, Mask: Data); // Write the Data to the LEDs
        }


    return 0;
}
```

Figure 25: Project Source File: Lab3_ledControl.c

15. Now build the application component using the hammer icon. Make sure to select your application as a component. Refer figure 26.

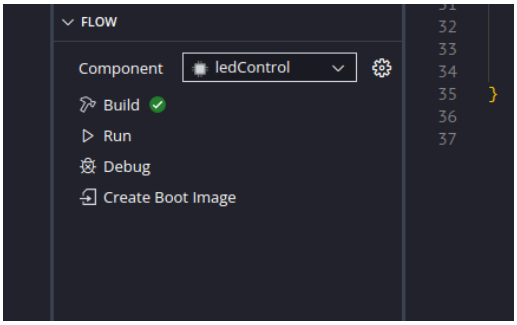


Figure 26: Building Application Component

Section 3: Running Application Project on Kria KV260

1. First connect the J2 Pmod pins to the LEDs with 200 Ohm registers according to circuit diagram in figure 27. There are total 12 pins in the Pmod. Pin numbers are mentioned in the PCB from 1 to 12. They are numbered in zigzag fashion. So all pins with even numbers are in one row and odd number pins on the other. The ground black wire is connected to the **Pin9**. After connection is complete the setup should look like figure 28. Here the red LED is the **LED-4** which corresponds to the **Most Significant Bit(MSB)** of the output data.

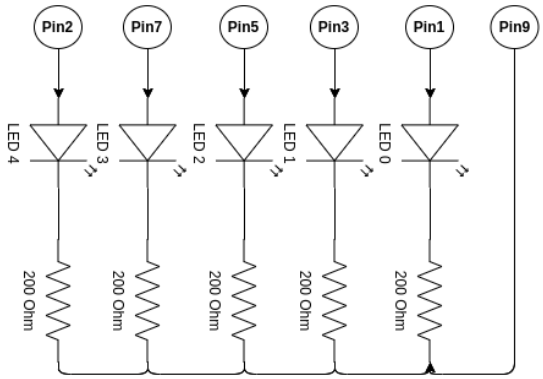


Figure 27: Circuit Diagram

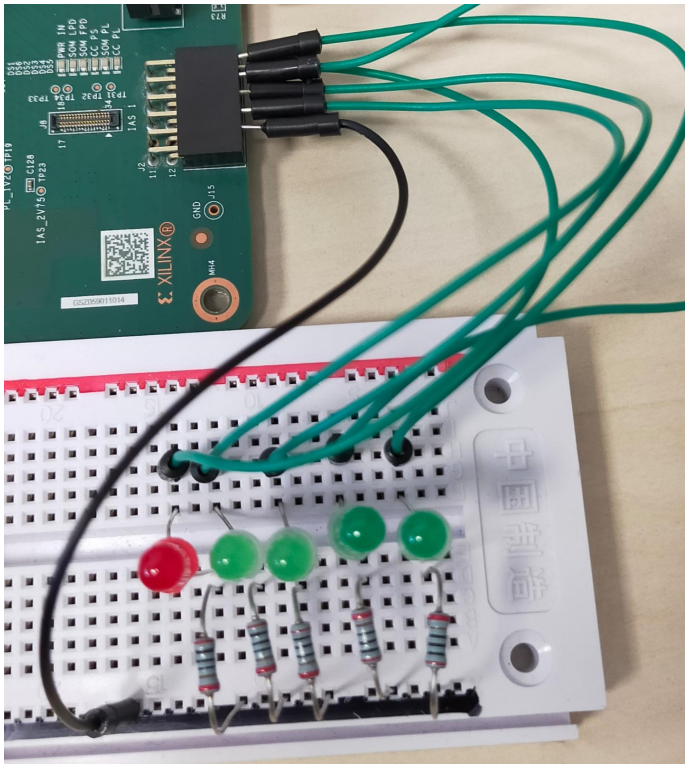


Figure 28: LED to PMOD Connection

2. Power the **Kria Board** and connect it's **Micro-USB** port to your machine. The **Micro-USB** port is used for UART communication to the **Kria Board** from your host machine similar to Lab1.

3. Open the **XSDB** console in Vitis. Refer figure 29.

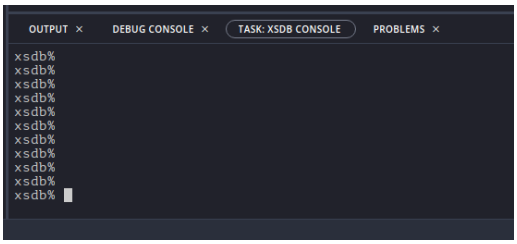


Figure 29: XSDB Console

4. Copy the following command in figure 30 from the url: https://xilinx.github.io/kria-apps-docs/creating_applications/2022.1/build/html/docs/baremetal.html#option-1-boot-using-jtag, on the **XSDB** console like figure 31. Then press **Enter**. This command will allow **Kria KV260** to boot from **JTAG**.

```
connect
targets -set -filter {name =~ "PSU"}
mwr 0xffca0010 0x0
mwr 0xff5e0200 0x0100
rst -system
```

Figure 30: XSDB Commands

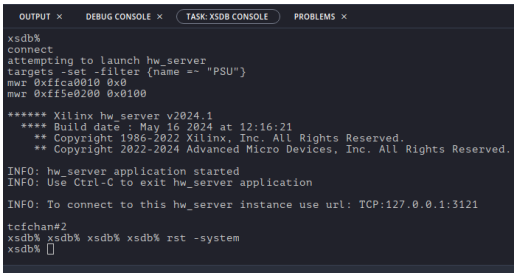


Figure 31: Changing the boot mode to JTAG via XSDB Console

5. Launch **PuTTY** on your host machine. Configure it like figure 32 for Serial connection at speed 115200 with correct **COM** port and click **Open**.
Note: You need to confirm on which **COM** port the board is connected. In this example it is connected as **ttyUSB1**.

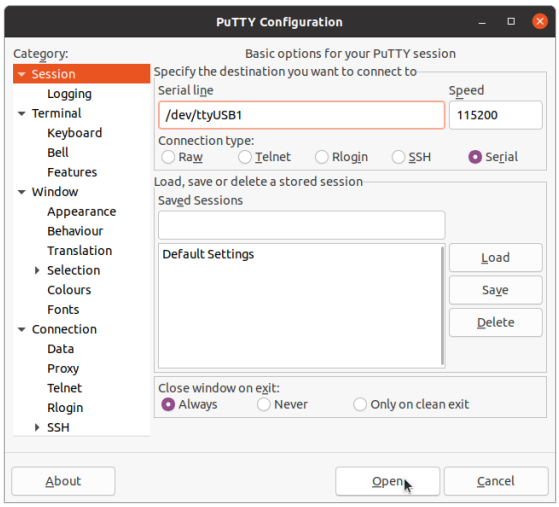


Figure 32: PuTTY Configuration

6. In the left-side Vitis **FLOW** window select the application component and the Click on **Run**. Refer figure 33.

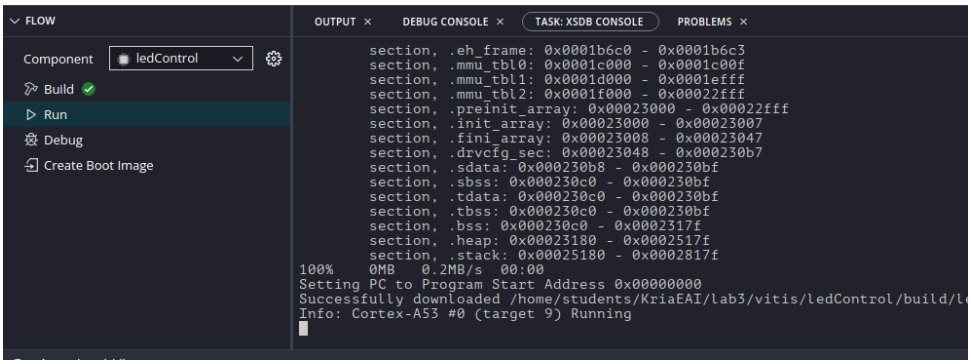


Figure 33: Running the Program

7. On successful run, you will see the "Enter the Data: "message on the **PuTTY** terminal as figure 34.

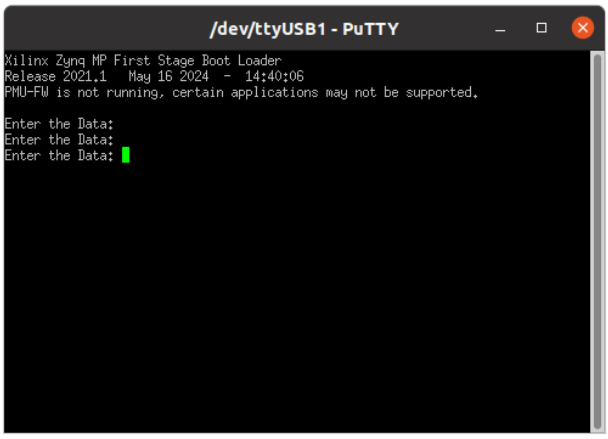


Figure 34: Putty Serial Console

8. Now if you Enter a number between 0 to 31 (5 bits)**(You maynot see the number you typed. Just type the number and hit Enter)**, you will see corresponding binary representation in the LEDs. Figure 35 is a demo output when 20(binary 10100) was entered in the console. Here the red LED is the most significant bit(MSB).

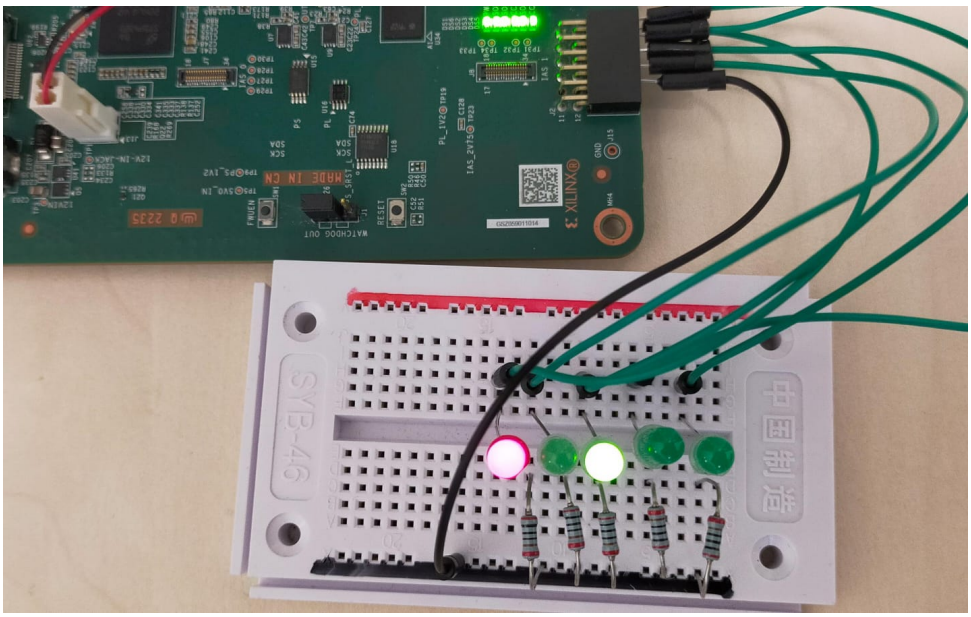


Figure 35: Output LED

Conclusion

Congratulations! Now you can successfully control the LEDs from the Terminal of the **Kria KV260 Board**. In this lab you created hardware platform in Vivado and imported it in Vitis to develop the software application. The C application takes the entered value from the PuTTY console and transfers the data to the LEDs connected to the PMODs of the **Kria KV260 Board**.